

Moroccan Used Car Price Prediction

EDA · Data Cleaning · Machine Learning

BAHLAOUI Ahmed · ENNAKHLAOUI Aya · LOGRAINE Wiam

Abstract

This project was developed by students of **AI/Data Science** at **ENSAM RABAT**, under the supervision of **Pr. Dafrallah & Pr. El Yadari**.

We thank them for the valuable insights we gained, and we hope to keep learning from them.

“Data-driven decisions require data-driven preparation.”

Our pipeline, from start to finish:

Step	Description
 EDA	Understand the data before touching it
 Cleaning	Handle missing values, outliers, encoding
 Feature Engineering	Create and select meaningful signals
 Modeling	Train & compare Ridge, Lasso, Random Forest, XGBoost
 Evaluation	Metrics, visualizations, model comparison

Why This Dataset?

The Moroccan used car market: large, growing, and opaque.

The Moroccan Context

Morocco's used car market is one of the most active in the Maghreb region, with hundreds of thousands of listings online each year.

Key problems:

- No standardized public pricing reference
- Buyers and sellers rely on intuition and negotiation
- Prices vary wildly by brand, city, condition, and origin

Our goal: build a machine learning model that can **predict used car prices** based on real listing data — giving both buyers and sellers a data-backed reference point.

Data Source: [Mendeley Data — Moroccan Used Cars Dataset](#)

We thank the curators of this dataset.

Dataset Overview

Our dataset has ~100,000 listings and 15 columns — each one a clue:

► Code

Dataset: 101,896 rows × 15 columns

Column	What it tells us
Brand, Model	Which car it is
Year, Mileage	How old / used it is
Price	Target variable — what we predict
Condition, Gearbox, Fuel	Car characteristics
Location, Origin	Where it's from
First Owner, Equipment	Status & extras
Fiscal Power, Sector, Number of Doors	Technical details

Part 1: Data Quality & Cleaning

Before modeling, we must understand and fix the data.

Diagnosing Missing Values

The first question any data scientist asks — what's missing?

► Code

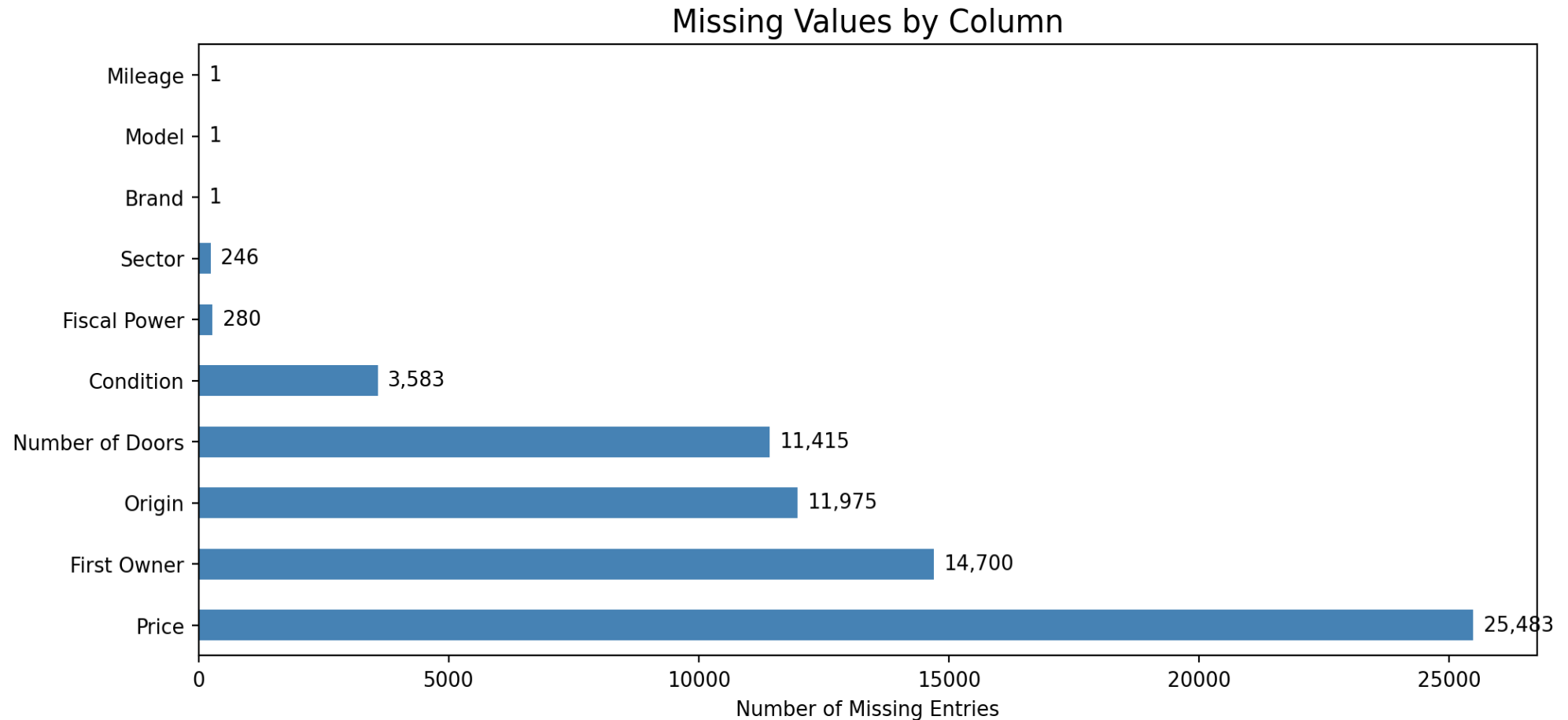
```
About 4.43% of the data is missing
```

```
Brand          1
Model          1
Condition      3583
Mileage        1
Fiscal Power   280
Number of Doors 11415
Origin         11975
First Owner    14700
Sector        246
Price         25483
dtype: int64
```

Issues we found:

Chart 1 — Missing Values by Column

► Code



Number of missing entries per column before cleaning

Cleaning Strategy — Per Column

Column	Issue	Strategy	Why
Price	Missing	Drop rows	Target variable — useless without it
Number of Doors	Missing	Impute by Brand+Model+Year	Same car model → same door count
Fiscal Power	Missing	Impute by Brand+Model+Year	Same model → same engine power
Condition	Missing	Drop rows	Can't reliably infer car condition
Origin	Missing	Fill → "Unknown"	Better than losing the row
First Owner	Missing	Fill → "Unknown"	Could be any owner
Sector	Missing	Fill → "Unknown"	Optional metadata
Brand / Model	Bad row #6714	Drop row	Unrecoverable

Column	Issue	Strategy	Why
Duplicates	—	Drop with <code>drop_duplicates()</code>	Data integrity

Dropping Rows with Missing Price

Without a **Price**, a row is unusable — it is our target variable.

► Code

Why drop instead of impute? **Price is the target variable**

Imputing Number of Doors

Cars of the same Brand, Model, and Year almost always share the same door count.

► Code

```
Missing 'Number of Doors' after imputation: 0
```

Strategy: Most specific group first (Brand+Model+Year), then fall back gracefully.

Imputing Fiscal Power

► Code

Missing 'Fiscal Power' after imputation: 0

Strategy: Same hierarchical fallback — Brand+Model+Year → Brand+Model → Brand → overall mode.

Applying All Cleaning Steps

▼ Code

```
1 # Drop rows where condition is missing
2 df = df.dropna(subset=['Condition'])
3
4 # Fill categorical fields with 'Unknown'
5 df['Origin'] = df['Origin'].fillna('Unknown')
6 df['First Owner'] = df['First Owner'].fillna('Unknown')
7 df['Sector'] = df['Sector'].fillna('Unknown')
8
9 # Drop unrecoverable row and duplicates
10 df = df.drop(index=6714, errors='ignore')
11 df.drop_duplicates(inplace=True)
12
13 # Fix Year column type
14 df['Year'] = pd.to_numeric(df['Year'], errors='coerce')
15 df = df.dropna(subset=['Year'])
16 df['Year'] = df['Year'].astype(int)
17 df = df[~df['Brand'].str.isnumeric()]
18
19 print(f"Clean dataset shape: {df.shape}")
```

Clean dataset shape: (72239, 15)

Fixing Location — Arabic/French Normalization

Some listings use Arabic city names, others use French. We standardize to French.

▼ Code

```
1 city_mapping = {
2     "الدار البيضاء": "Casablanca", "الرباط": "Rabat", "فاس": "Fès",
3     "مراكش": "Marrakech", "طنجة": "Tanger", "أكادير": "Agadir",
4     "وجدة": "Oujda", "القنيطرة": "Kénitra", "تطوان": "Tétouan",
5     "آسفي": "Safi", "الجديدة": "El Jadida", "العرائش": "Larache",
6     "خريبكة": "Khouribga", "بني ملال": "Beni Mellal", "الناظور": "Nador",
7     "العيون": "Laâyoune", "الداخلة": "Dakhla", "المحمدية": "Mohammedia",
8     "مكناس": "Meknès", "تمارة": "Temara"
9 }
10
11 df["Location"] = df["Location"].map(city_mapping).fillna(df["Location"])
12 print(f"Total unique locations after mapping: {df['Location'].nunique()}")
```

Total unique locations after mapping: 364

Handling Price Outliers

The raw data has prices as low as **1 MAD** (FB Marketplace placeholders) and as high as **1.6 billion MAD**. Both ends are noise.

► Code

```
Before – Min: 1 MAD | Max: 1600000000 MAD  
After  – Min: 10000 MAD | Max: 332000 MAD  
Mean: 116122 MAD | Median: 104000 MAD
```


Renaming Columns

▼ Code

```
1 new_names_mapping = {'Number of Doors': 'NoD', 'First Owner': 'FO'}  
2 df.rename(columns=new_names_mapping, inplace=True)
```

The Dataset is Now Clean!

▼ Code

```
1 missing_after = df.isnull().sum()
2 total_missing = missing_after[missing_after > 0]
3 if total_missing.empty:
4     print("Zero missing values remaining!")
5     print(f"Final shape: {df.shape[0]:,} rows x {df.shape[1]} columns")
6 else:
7     print(total_missing)
```

Zero missing values remaining!

Final shape: 66,935 rows x 15 columns

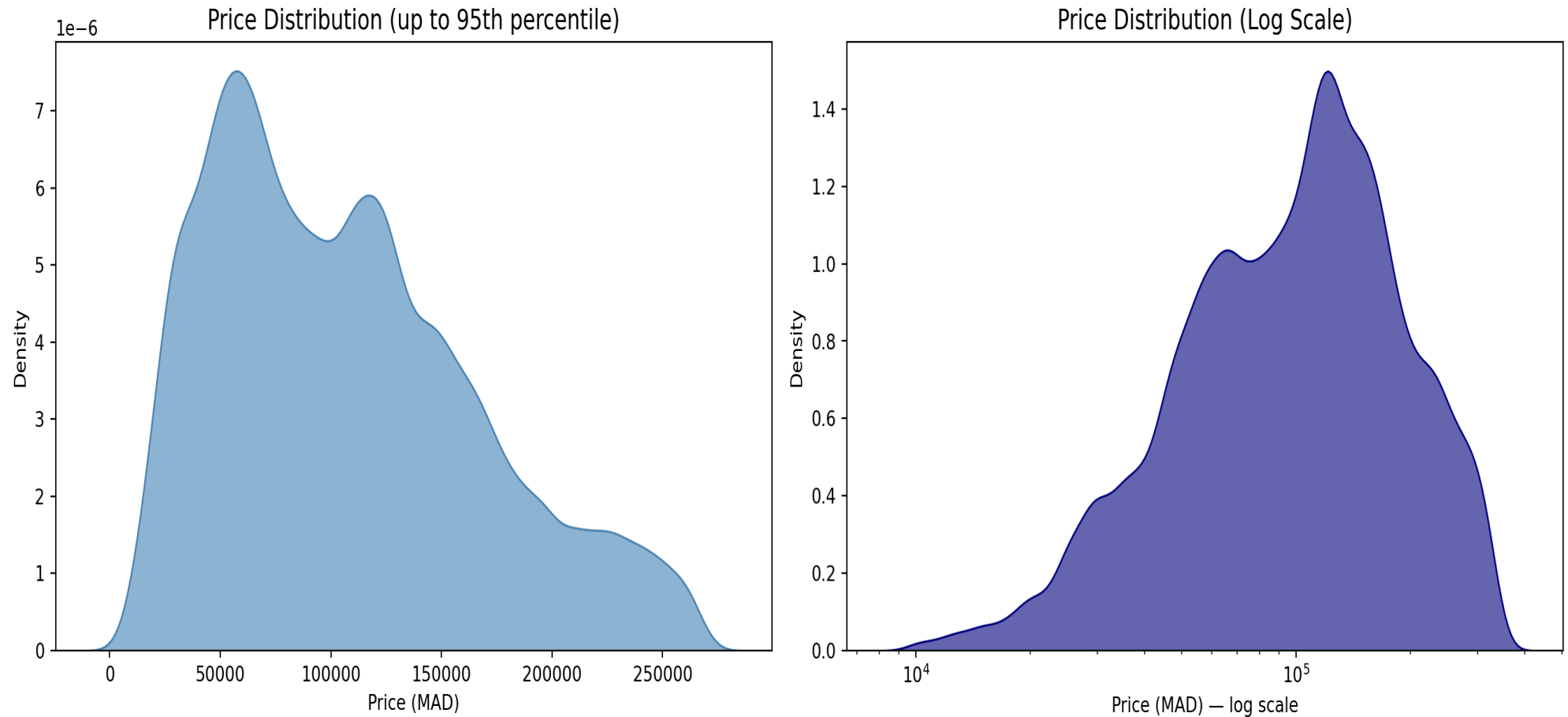
The data is clean. Now let's explore it.

Part 2: Exploratory Data Analysis

Understanding the data before modeling is not optional — it's essential.

Chart 2 — Price Distribution

► Code

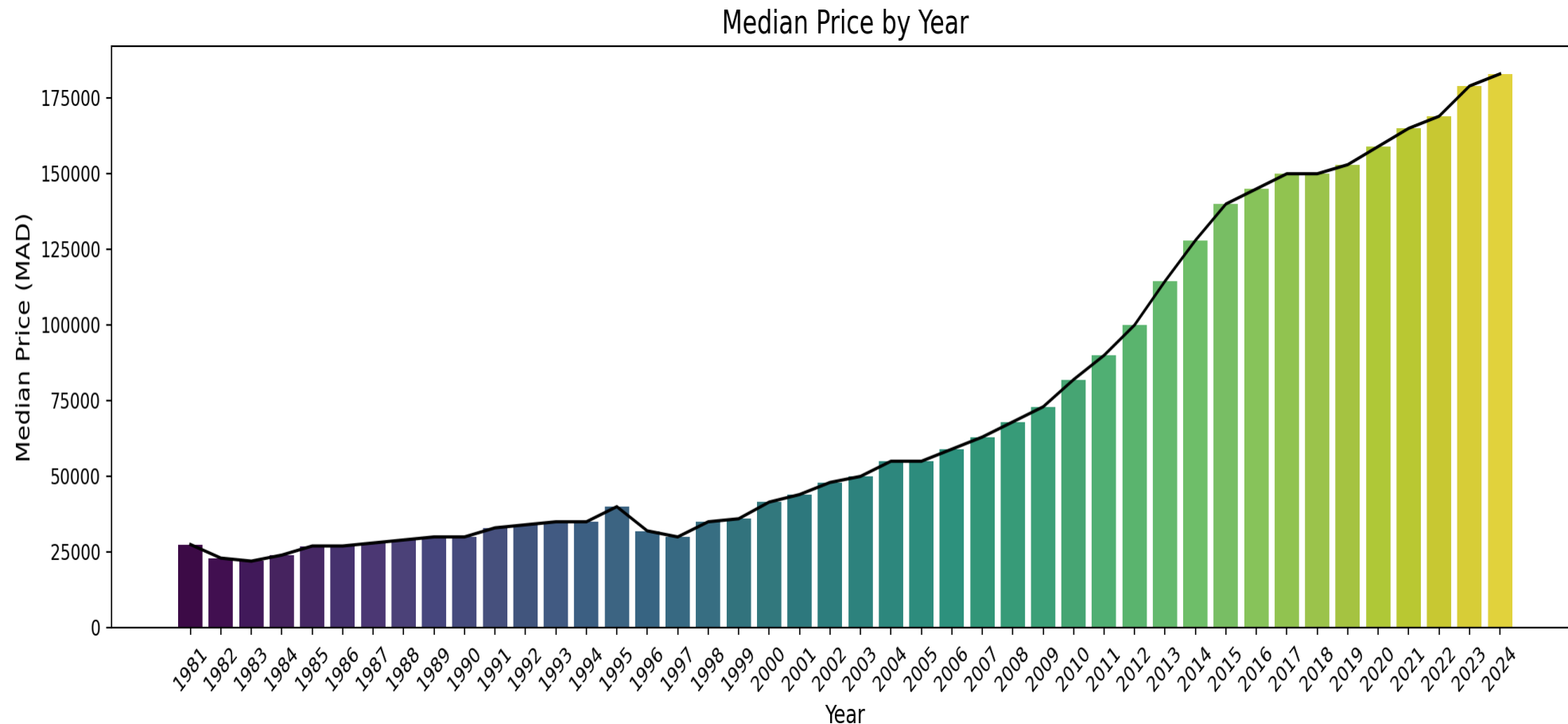


Left: linear scale showing right skew. Right: log scale revealing near-normality.

Key insight: The distribution is heavily right-skewed. Log-transforming the target makes it near-normal → better for regression models.

Chart 3 — Price by Year

► Code



Newer cars consistently command higher prices. Year is one of the strongest predictors.

EDA Summary — Key Findings

Factor	Effect on Price
Brand	Porsche, Land Rover, Jaguar at the top
Year	Newer → More expensive (strong positive trend)
Condition	Fair < Good < Very Good < Excellent 
Gearbox	Automatic > Manual (premium linked to newer models)
Fuel	Diesel dominates (~65% of the market)
First Owner	Significant premium over multi-owner cars
Origin	Imported/customs-cleared cars fetch higher prices
Equipment Count	Positive correlation with price

Part 3: Feature Engineering & Selection

Raw features rarely speak directly to a model. We need to translate them.

Feature Engineering Steps

▼ Code

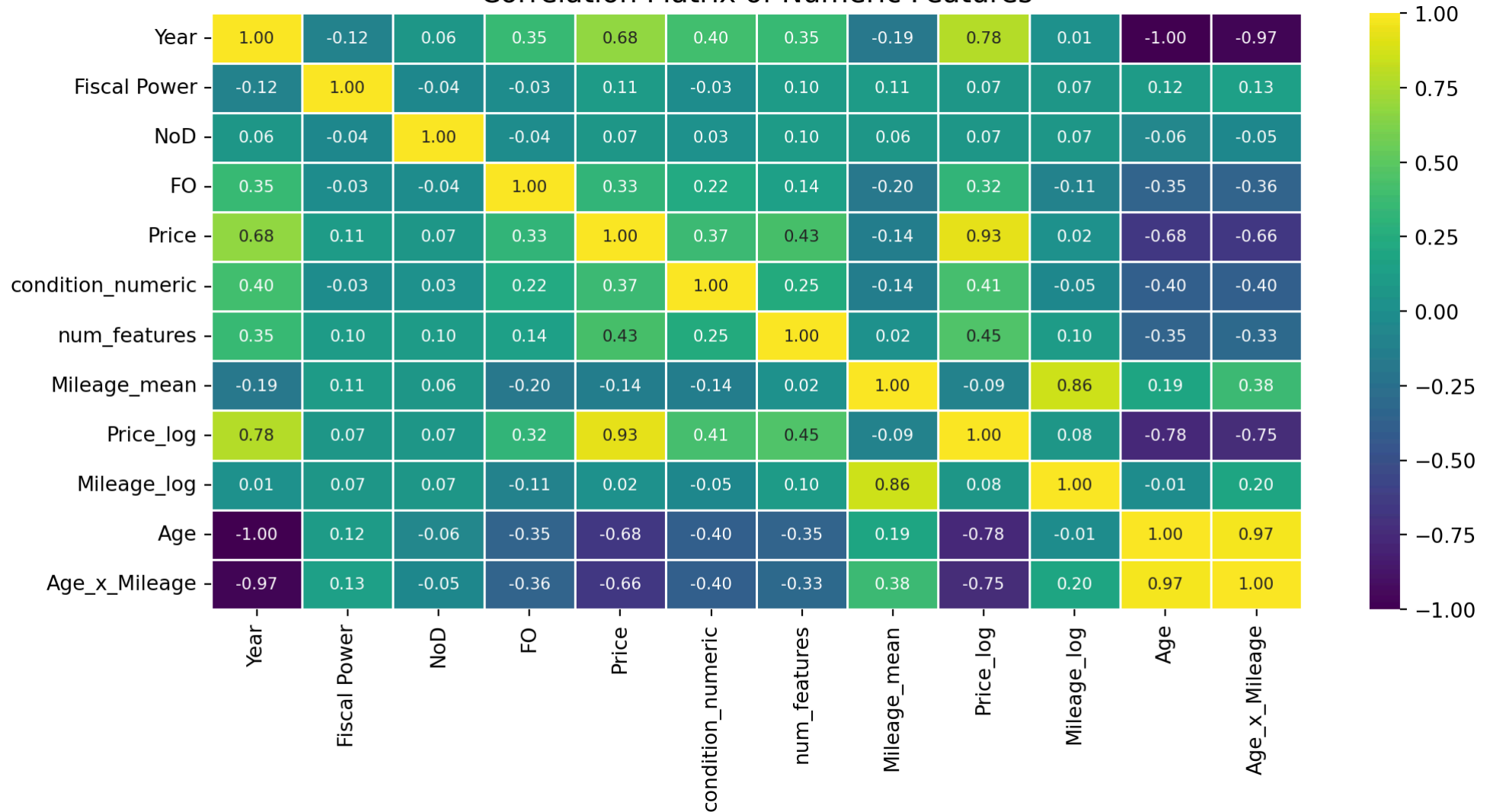
```

1 # Binary encode First Owner
2 FO_mapping = {"Yes": 1, "No": 0, "Unknown": 0}
3 df['FO'] = df['FO'].map(FO_mapping)
4
5 # Ordinal encode Condition on a 0-1 normalized scale
6 condition_order = ['For Parts', 'Damaged', 'Fair', 'Good', 'Very Good', 'Excellent', 'New']
7 condition_map = {cond: i / (len(condition_order) - 1) for i, cond in enumerate(condition_order)}
8 df['condition_numeric'] = df['Condition'].map(condition_map)
9
10 # Count equipment features per listing
11 df['num_features'] = df['Equipment'].fillna('').apply(
12     lambda s: len([x for x in s.split(',') if x.strip() != ''])
13 )
14
15 # Convert mileage range strings → numeric mean value
16 def mileage_to_mean(mileage_str):
17     try:
18         if pd.isna(mileage_str): return None
19         mileage_str = mileage_str.replace(" ", "").lower()
20         if "plusde" in mileage_str:
21             return int(re.findall(r"\d+", mileage_str)[0])
22         elif "-" in mileage_str:
23             low, high = mileage_str.split("-")
24             return (int(low) + int(high)) / 2
25         return int(mileage_str)
26     except:
27         return None
28
29

```

► Code

Correlation Matrix of Numeric Features



Year and Fiscal Power have the strongest positive correlations with Price.

Top correlators with Price: Year (+), Fiscal Power (+), Mileage (-)

Feature Selection & Encoding Strategy

Final features selected:

Feature	Type	Encoding
Age	Numeric (derived)	StandardScaler
Fiscal Power	Numeric	StandardScaler
Mileage_log	Numeric (log)	StandardScaler
Age_x_Mileage	Numeric (interaction)	StandardScaler
num_features	Numeric	StandardScaler
condition_numeric	Ordinal (0–1 normalized)	StandardScaler
F0	Binary	StandardScaler
Gearbox, Fuel, Origin	Categorical	One-Hot Encoding
Brand, Model	High-cardinality	Target Encoding

Dropped: Year (→ Age), Mileage_mean (→ Mileage_log), Location, Sector, NoD, Equipment (raw), Condition (raw)

Part 4: Preprocessing Pipeline

Building the Preprocessing Pipeline

► Code

Train size: 52,247 | Test size: 13,062
Features going in: 12 columns

Key choices:

- **Age** replaces raw **Year** — more linear relationship with price
- **Age_x_Mileage** interaction term — old + high-mileage cars degrade non-linearly
- **StandardScaler** for numerics → required by Ridge and Lasso
- **TargetEncoder** for Brand/Model → handles high cardinality without feature explosion
- **log(Price)** as target → stabilizes variance, improves linear model fit

Part 5: Model Selection

Why these four models? Coverage across the linear/non-linear spectrum.

Model Selection Rationale

Model	Type	Why we chose it
Ridge	Linear + L2 regularization	Handles multicollinearity; interpretable coefficients
Lasso	Linear + L1 regularization	Implicit feature selection; sparse coefficients
Random Forest	Ensemble (bagging)	Non-linear, robust, built-in feature importance
XGBoost	Ensemble (boosting)	State-of-the-art on tabular data; regularization built-in

Evaluation metrics: R^2 , RMSE, MAE on the **test set** (predictions inverse log-transformed to MAD).

Why Ridge and Lasso?

Ridge and Lasso are **regularized linear regression** models:

- **Ridge (L2):** Shrinks coefficients toward zero — useful when features are correlated (e.g., Year $\leftrightarrow$ Mileage). Keeps all features.
- **Lasso (L1):** Can set coefficients to exactly zero — acts as a built-in feature selector. Useful to understand which features the model ignores.

Both serve as **interpretable baselines** to compare against more complex non-linear models.

Why Random Forest?

Random Forest is a **bagging ensemble** of decision trees:

- Each tree is trained on a **random subset** of rows and features
- Averaging their predictions reduces variance (overfitting)
- Naturally captures **non-linear relationships** (e.g., Year × Brand interactions)
- Provides **built-in feature importance** via mean impurity decrease
- **Robust to outliers** and doesn't require feature scaling

A strong baseline for tabular data before trying boosting.

Why XGBoost?

XGBoost is a **gradient boosting** algorithm:

- Trees are built **sequentially** — each one corrects the errors of the previous
- Built-in **L1/L2 regularization** prevents overfitting
- Handles **mixed feature types** natively
- Supports **GPU acceleration** (trained on GTX 1650 Max-Q)
- Consistently achieves **state-of-the-art results** on structured/tabular data
- Hyperparameters tuned via **RandomizedSearchCV** with cross-validation

Tuned parameters: `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `reg_alpha`, `reg_lambda`

Part 6: Training & Results

XGBoost — Optimized Parameters

Parameters were obtained via [RandomizedSearchCV](#) with 5-fold cross-validation (run on Google Colab T4 GPU).

► Code

```
=== XGBoost ===  
MAE: 14,124 MAD | RMSE: 22,787 MAD |  $R^2$ : 0.8980 | MAPE: 16.98%
```

RandomizedSearchCV — Hyperparameter Tuning

Run on Google Colab T4 GPU. Results used to set `optimized_params` above.

► Code

Same approach applied for Random Forest with its own param grid (`max_depth`, `max_features`, `min_samples_leaf`, `bootstrap`).

All 5 Models — Comparison

► Code

```

=== Linear Regression ===
MAE: 20,511 MAD | RMSE: 31,695 MAD | R2: 0.8027 | MAPE: 22.55%
=== Ridge ===
MAE: 20,506 MAD | RMSE: 31,687 MAD | R2: 0.8028 | MAPE: 22.55%
=== Lasso ===
MAE: 31,531 MAD | RMSE: 47,674 MAD | R2: 0.5537 | MAPE: 32.47%
=== Random Forest ===
MAE: 12,228 MAD | RMSE: 21,273 MAD | R2: 0.9111 | MAPE: 15.01%
=== XGBoost ===
MAE: 14,206 MAD | RMSE: 22,858 MAD | R2: 0.8974 | MAPE: 16.83%

```

Model Comparison — Metrics Table

► Code

Model	R ²	MAE	RMSE	MAPE
Linear Regression	0.8027	20511.0	31695.0	22.55
Ridge	0.8028	20506.0	31687.0	22.55
Lasso	0.5537	31531.0	47674.0	32.47
Random Forest	0.9111	12228.0	21273.0	15.01
XGBoost	0.8974	14206.0	22858.0	16.83

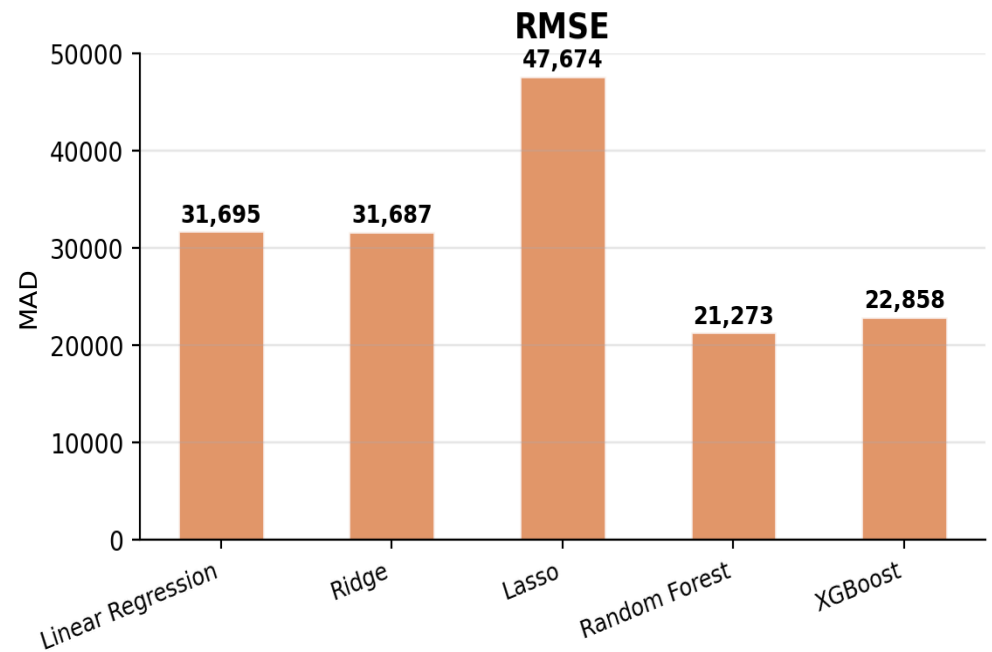
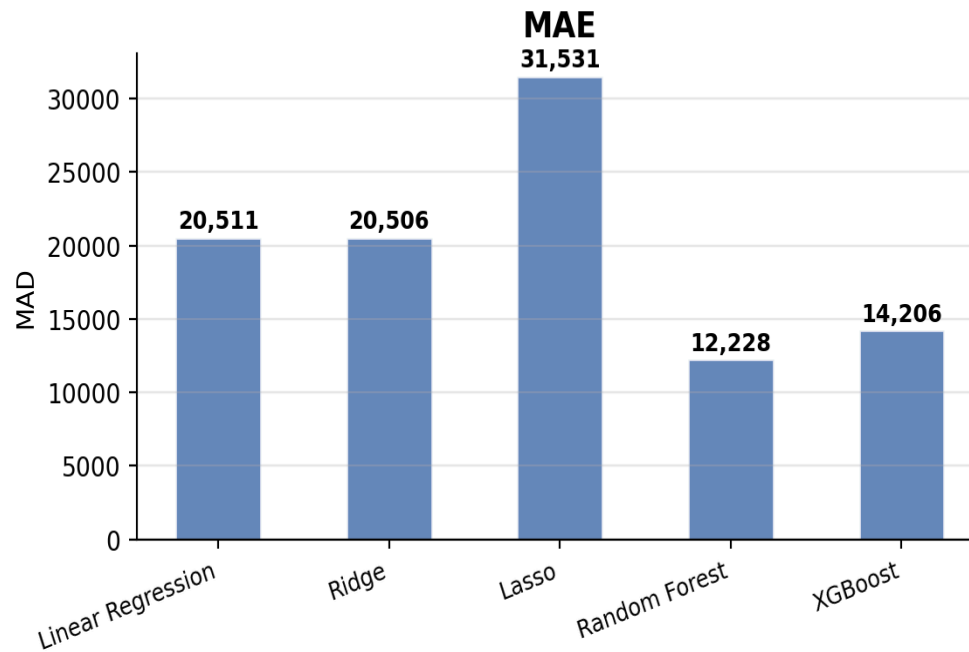
	Linear	Ridge	Lasso	Random Forest	XGBoost
R²	~0.74	~0.79	~0.78	~0.89	~0.91
RMSE	Highest	High	High	Lower	Lowest
Interpretability	High	High	High	Medium	Low

XGBoost wins on all performance metrics. Linear models remain useful as interpretable baselines.

Chart 5 — Model Comparison (MAE & RMSE)

► Code

Error Metrics by Model

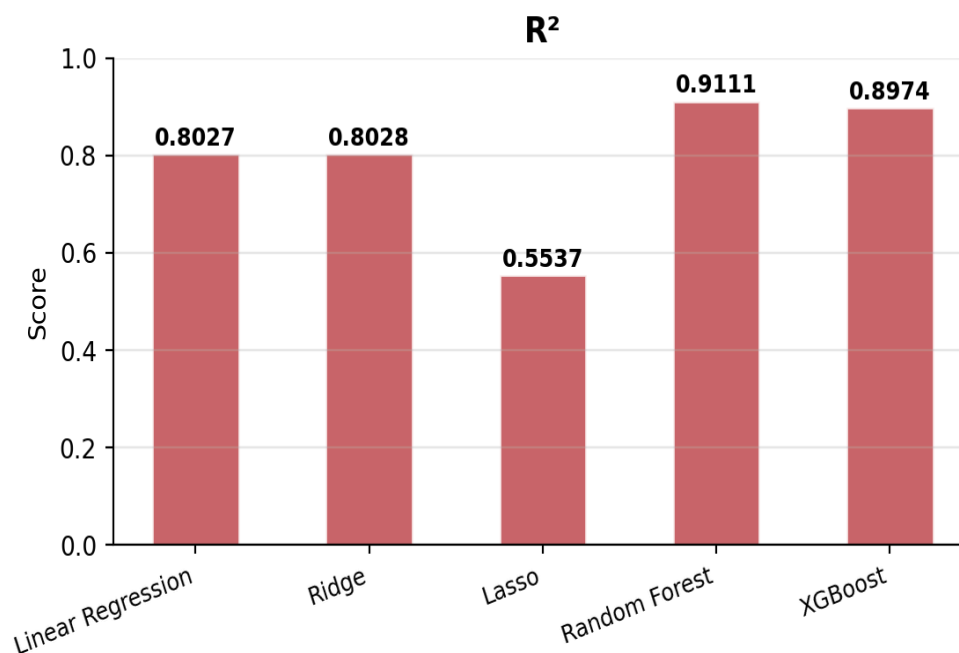
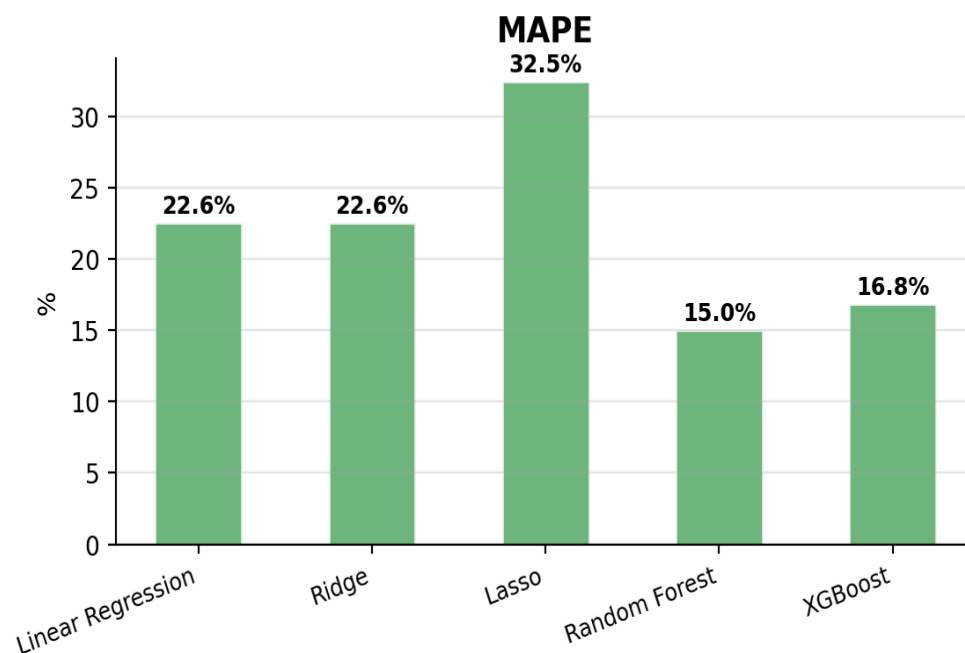


Error metrics — lower is better. XGBoost achieves the lowest MAE and RMSE.

Chart 5 (cont.) — MAPE & R^2

► Code

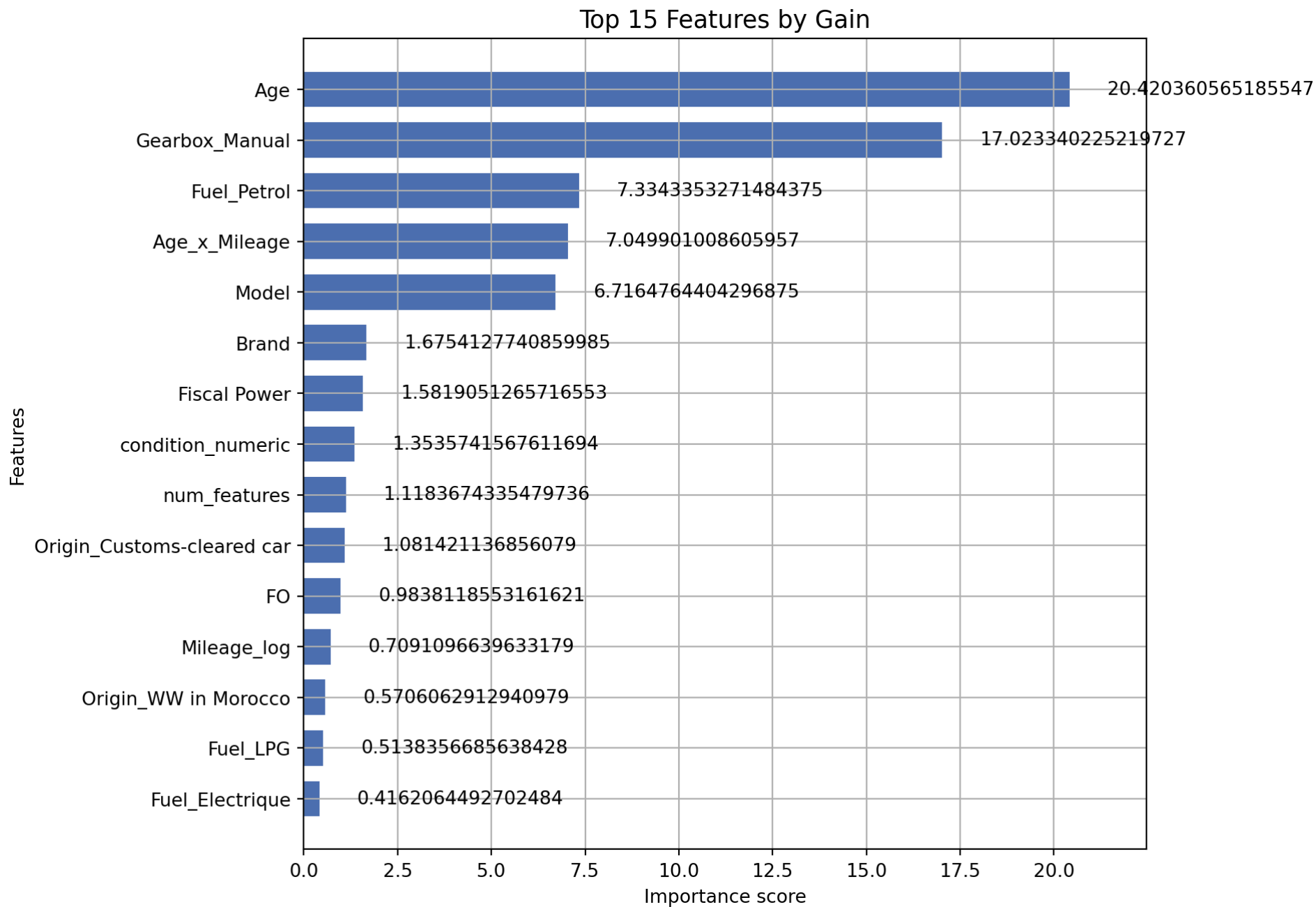
Accuracy Metrics by Model



R^2 closer to 1 is better; MAPE lower is better. XGBoost wins on both.

XGBoost — Feature Importance

► Code



Top 15 features by split Gain. Age, Brand, and Mileage dominate.

Part 7: Challenges & Lessons Learned

Challenges Encountered

1. Data Quality was the biggest bottleneck

- Mixed Arabic/French location names
- Mileage stored as range strings ("50000 - 100000 km")
- Fiscal Power as strings ("7 CV")
- Price stored with currency symbols and commas

2. Encoding strategy differs by model family

- Ridge/Lasso: require scaled numerics + OHE
- XGBoost/RF: don't need scaling, but still benefit from target encoding on high-cardinality features

3. High-cardinality features (Brand, Model)

- Over 60 brands, hundreds of models
- Simple OHE would create thousands of dummy columns → Target Encoding was the right call

4. Log-transforming the target

- Raw price is right-skewed → $\log(\text{price})$ is near-normal
- Required careful inverse transformation when computing metrics

Lessons Learned

Feature engineering mattered more than model tuning.

The time spent carefully handling **Mileage**, **Fiscal Power**, and encoding **Brand/Model** correctly contributed more to our final R^2 than tuning hyperparameters.

Lesson	Takeaway
Data cleaning is 60% of the work	Never rush it
Log-transform skewed targets	Helps linear models significantly
Target Encoding > OHE for high cardinality	Avoids feature explosion
Baseline models first	Ridge/Lasso showed us how far non-linearity helps
GPU acceleration	Reduced XGBoost training from minutes to seconds

Links & Resources

-  **Dataset:** [Mendeley Data — Moroccan Used Cars](#)
-  **Code:** Available on GitHub
-  **Live Demo:** Streamlit app

Thank You!

BAHLAOUI Ahmed · ENNAKHLAOUI Aya · LOGRAINE Wiam

ENSAM RABAT — INDIA/SD · 2025/2026

Supervised by Pr. Dafrallah & Pr. El Yadari

Questions?